

Proyecto 1

1. Algoritmo de Solución

a. Grafo:

El problema se modela como un grafo dirigido y ponderado. Cada posición del campo, identificada por su órbita y su fase (órbita, posición), se representa como un vértice único. Para evitar trabajar directamente con una matriz de $N \times M$, cada par (órbita, posición) (indexado desde 1) se convierte en un único índice lineal mediante la fórmula $\text{nodo} = (\text{órbita}-1) \cdot M + (\text{posición}-1)$, obteniendo así un arreglo de vértices numerados de 0 a $N \cdot M - 1$. Esta transformación permite representar el grafo con una lista de adyacencia indexada por enteros, más eficiente en tiempo y memoria que manejar una matriz bidimensional, dado que cada vértice tiene muy pocos vecinos (el grafo es disperso).

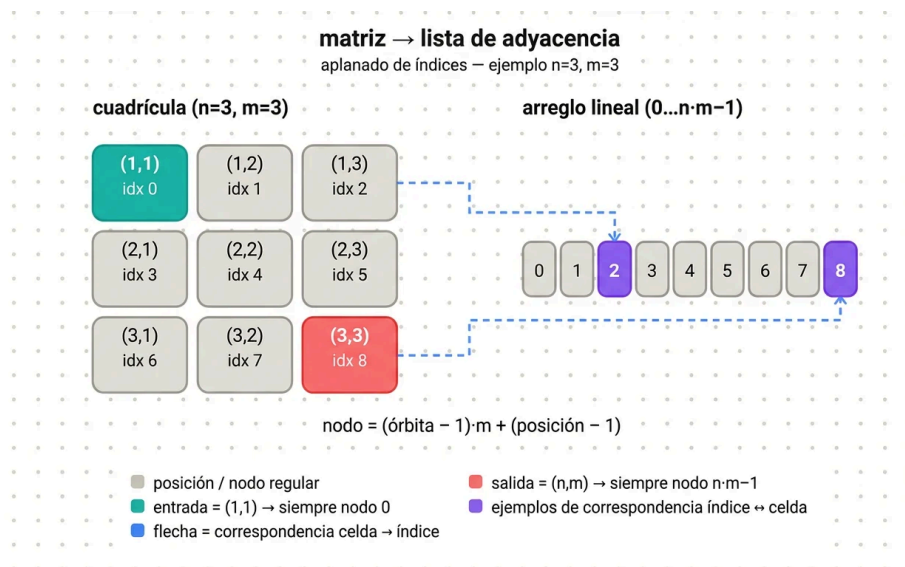


Figura elaborada por los autores con apoyo de IA para su diseño visual, a partir del modelo ya descrito.

Sobre este conjunto de vértices se construyen dos tipos de arista, reflejando las dos formas de movimiento descritas en el enunciado:

- Movimiento dentro de una órbita: entre posiciones adyacentes de una misma órbita, en ambos sentidos, con peso igual al costo de energía fijo de esa órbita (e_i).
- Portales: aristas dirigidas de una posición específica de una órbita hacia una posición de otra órbita de índice mayor, con peso 0, ya que no consumen energía y solo se recorren en un sentido.

Esta reducción del problema físico a un problema de camino mínimo en un grafo ponderado permite resolverlo con un algoritmo clásico de caminos de costo mínimo (Dijkstra), en lugar de diseñar un algoritmo ad-hoc para el dominio del enunciado.

Validación:

Se verificó que construirGrafo no requiere manejo especial para los casos borde: cuando $p=0$ simplemente no se agregan aristas de portal, y cuando $n=1$ el término puede llegar del inicio al final moviéndose únicamente dentro de la única órbita disponible, sin depender de portales. En el caso extremo $n=1$, $m=1$, el nodo de entrada coincide con el de salida y no se generan aristas, lo cual es correcto pues no se requiere movimiento alguno.

b. Justificación de selección de algoritmo:

Se utilizó Dijkstra ya que tiene una aplicación válida debido a que todas las aristas del grafo tienen pesos no negativos. Las aristas que corresponden al movimiento entre posiciones adyacentes tienen peso e_i , que es equivalente al consumo de energía de la órbita, mientras que los portales tienen peso 0. Esta propiedad garantiza que cuando el algoritmo selecciona el vértice no procesado con menor distancia acumulada, esa distancia puede considerarse definitiva. Cualquier otro camino alternativo descubierto después tendría que agregar únicamente aristas de costos mayores o de costo 0, por lo que no se podría generar una distancia menor. Siendo así, Dijkstra permite obtener de manera correcta el mínimo consumo de energía necesario para llegar desde la entrada hasta la salida

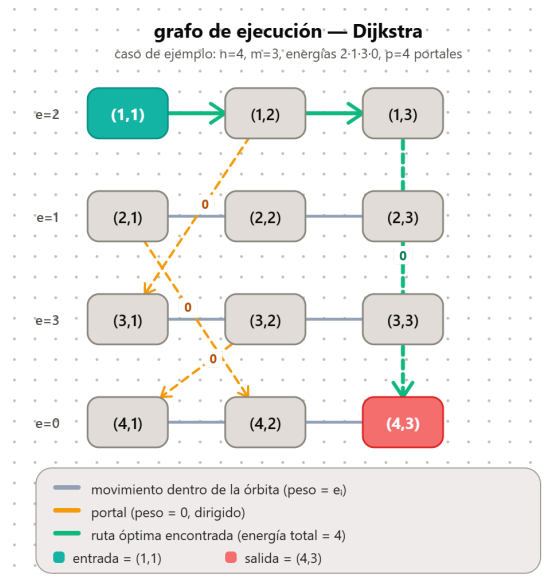


Figura elaborada por los autores. La representación visual se generó con apoyo de IA a partir del caso de ejemplo del enunciado y el grafo descrito anteriormente.

La Figura ilustra el grafo construido por construirGrafo para el caso de ejemplo del enunciado ($n=4$ órbitas, $m=3$ posiciones, energías 2, 1, 3 y 0, con 4 portales). Las aristas grises representan el movimiento entre posiciones adyacentes dentro de una misma órbita, con peso igual a la energía e_i de esa órbita. Las aristas punteadas en amarillo son los portales

que no forman parte de la ruta óptima, dirigidos hacia una órbita de índice mayor y con peso 0. El portal $(1,3) \rightarrow (4,3)$, al ser parte de dicha ruta, se resalta en verde junto con el resto del camino. Sobre este grafo, la ruta resaltada en verde, $(1,1) \rightarrow (1,2) \rightarrow (1,3) \rightarrow \text{portal} \rightarrow (4,3)$, es la que nuestra implementación de Dijkstra devuelve como de menor energía, con un costo total de $2+2+0=4$, coincidiendo con el resultado esperado en P1.out.

c. Justificación de min-heap:

Dijkstra requiere seleccionar repetidamente el vértice no procesado que tiene la menor distancia acumulada. Realizar esta búsqueda recorriendo todos los nodos tendría un costo de $O(V)$ por selección y complejidad de $O(V^2)$. Para evitar esto se usa min-heap mediante un PriorityQueue que mantiene los nodos ordenados según su distancia conocida. Así extraer el nodo con menor distancia e insertar una nueva alternativa requiere $O(\log V)$. Combinado con la representación con listas de adyacencia, esto permite que se ejecute Dijkstra en $O((V+E)\log V)$.

2. Análisis de Complejidades

a. Variables de tamaño:

- n : número de órbitas.
- m : número de posiciones por órbita.
- p : número de portales.
- $V = n \cdot m$: número total de vértices del grafo.
- E : número total de aristas, acotado por $2 \cdot n \cdot (m-1) + p$, es decir, $O(n \cdot m + p)$.

b. Complejidad temporal:

La construcción del grafo (inicializar las $n \cdot m$ listas de adyacencia, recorrer cada órbita agregando sus aristas de movimiento, y recorrer los p portales) toma $O(n \cdot m + p)$, ya que cada vértice y cada arista se procesan una sola vez. Posteriormente, Dijkstra utilizando una lista de adyacencia y un min-heap requiere de $O((V+E)\log V)$ y que en tamaño de variables es $O((nm+p)\log(nm))$. Dijkstra es quien domina la complejidad temporal y por ende esta es $O(nm+p) \cdot (\log(nm))$.

c. Complejidad espacial:

El grafo ocupa $O(n \cdot m + p)$, correspondiente a las listas de adyacencia y sus aristas. A esto se suma el espacio usado por Dijkstra para las distancias y la cola de prioridad, también $O(n \cdot m)$. Además, Dijkstra usa un arreglo de distancias y un arreglo de visitados de tamaño V junto con un min-heap que en el peor caso puede almacenar múltiples entradas generadas durante las relajaciones requerimiento hasta $O(E)$. Por lo que la complejidad espacial total es $O(V+E)$, que es $O(nm+p)$.